Class: Algorithms & Data Structures Semester: Spring 2025

Noah Wiley
noahwiley@mail.fresnostate.edu
Lab Number - Section 09

CSCI 115 Lab 2 Report; Fibonacci Numbers

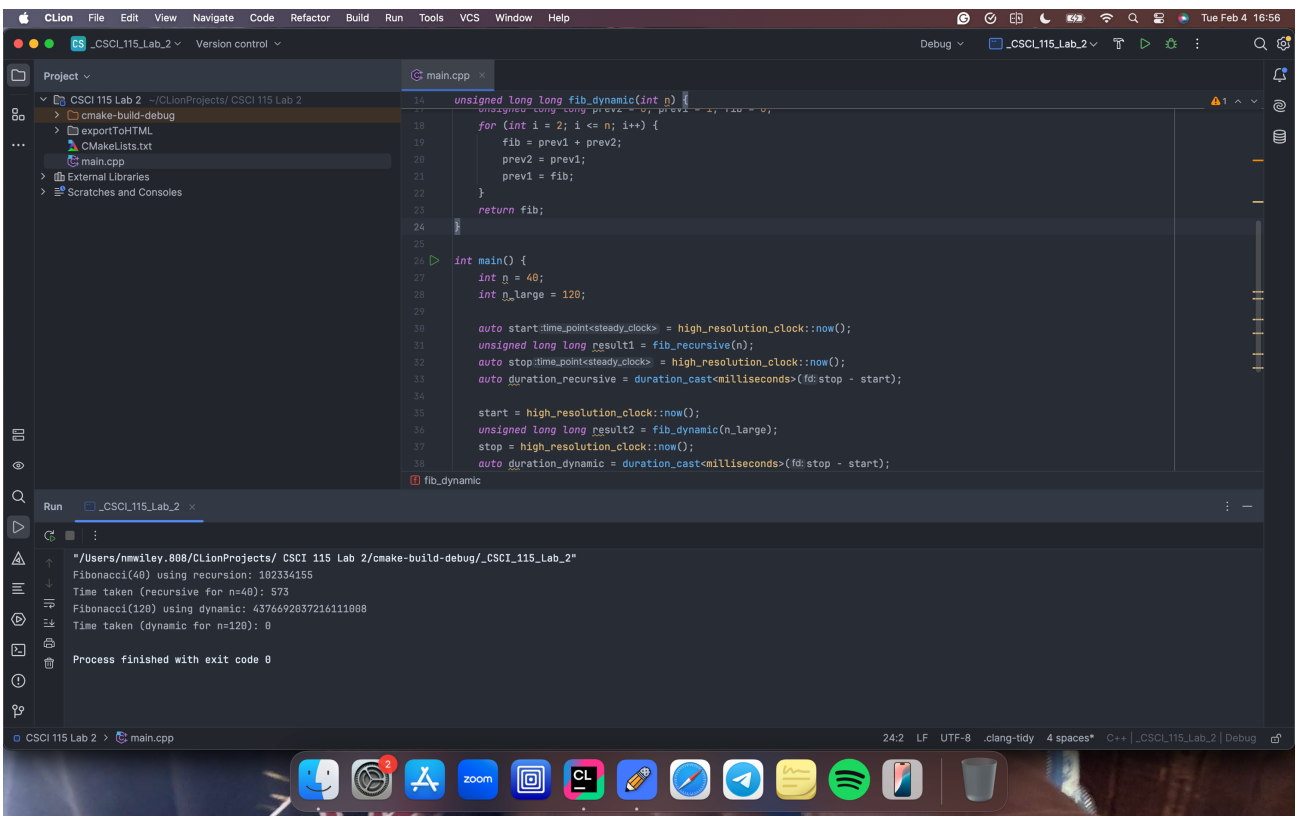
- Statement of Objectives
  - This lab aims to analyze different methods for computing Fibonacci numbers and evaluate their efficiency. Computing Fibonacci numbers efficiently is a fundamental problem in computer science, often used approaches to illustrate different algorithmic paradigms. A naive recursive approach follows the direct definition of the sequence but can be highly inefficient due to redundant calculations. In contrast, optimized approaches such as dynamic programming or iterative methods reduce unnecessary computations and significantly improve performance. The objective of this lab is to implement a basic recursive algorithm for Fibonacci computation in C++. This report details the experimental procedure, analysis, encountered challenges, and conclusions drawn from this experiment.

- Experimental Procedure:
  - To achieve the lab objectives, the following procedure was followed:
    - Naive Recursive Method:
      - Implement a recursive function in C++ that directly follows the Fibonacci definition.
      - Use the chrono library to measure execution time for F(40) and F(120).
    - Optimized Approach (Dynamic Programming or Iterative Method):
      - Implement a more efficient method using either memoization or an iterative approach.
      - Measure execution time for F(40) and F(120).
    - Performance Analysis:
      - Compare execution times between the two methods.
      - Evaluate the computational complexity of each approach.

- Analysis:
  - The following results were obtained from the experiment:
    - Initially, we attempted to compute F(120) using both methods, but naive recursive approach proved infeasible due to excessive computation time. Furthermore, during the testing, the dynamic approach produced an incorrect result for F(120), likely due to an implementation error in handling large numbers or incorrect indexing. To ensure reliable results and a fair comparison, we instead focused on F(40), where the correctness of both methods could be verified.
    - The naive recursive approach has an exponential time complexity O(2^n), making it impractical for large values of n. In the experiment, computing F(40) took 573

milliseconds, highlighting the inefficiency of recursion due to repeated calculations. On the other hand, the optimized iterative approach computed F(40) in 0 milliseconds, demonstrating its efficiency in reducing redundant operations.

- The optimized approach drastically reduces computation time, often achieving linear O(n) or logarithmic O(log n) complexity, depending on the implementation.



- Encountered Problems:
  - The only error I encountered was the recursive method was highly inefficient, leading to long execution times and potential stack overflow errors. This was resolved by implementing memoization and an iterative approach.

- Conclusions:
  - Through this lab, the following concepts were reinforced:
    - The naive recursive algorithm is highly inefficient due to redundant calculations
    - Optimized approaches like dynamic programming or iteration significantly reduce execution time and improve performance.
    - Computing large Fibonacci numbers requires careful handling of data types to prevent overflow or inaccuracies.
    - Efficient algorithms are essential for computing large Fibonacci numbers within a reasonable timeframe.
    - The lab provided insight into algorithm optimization and its impact on computational efficiency.

        - This experiment highlighted the contrast between inefficient and optimized solutions and provided hands-on experience in performance analysis using C++.

- References:
    - Lecture Slides On Fibonacci Numbers